

PROGRAMACIÓN AVANZADA

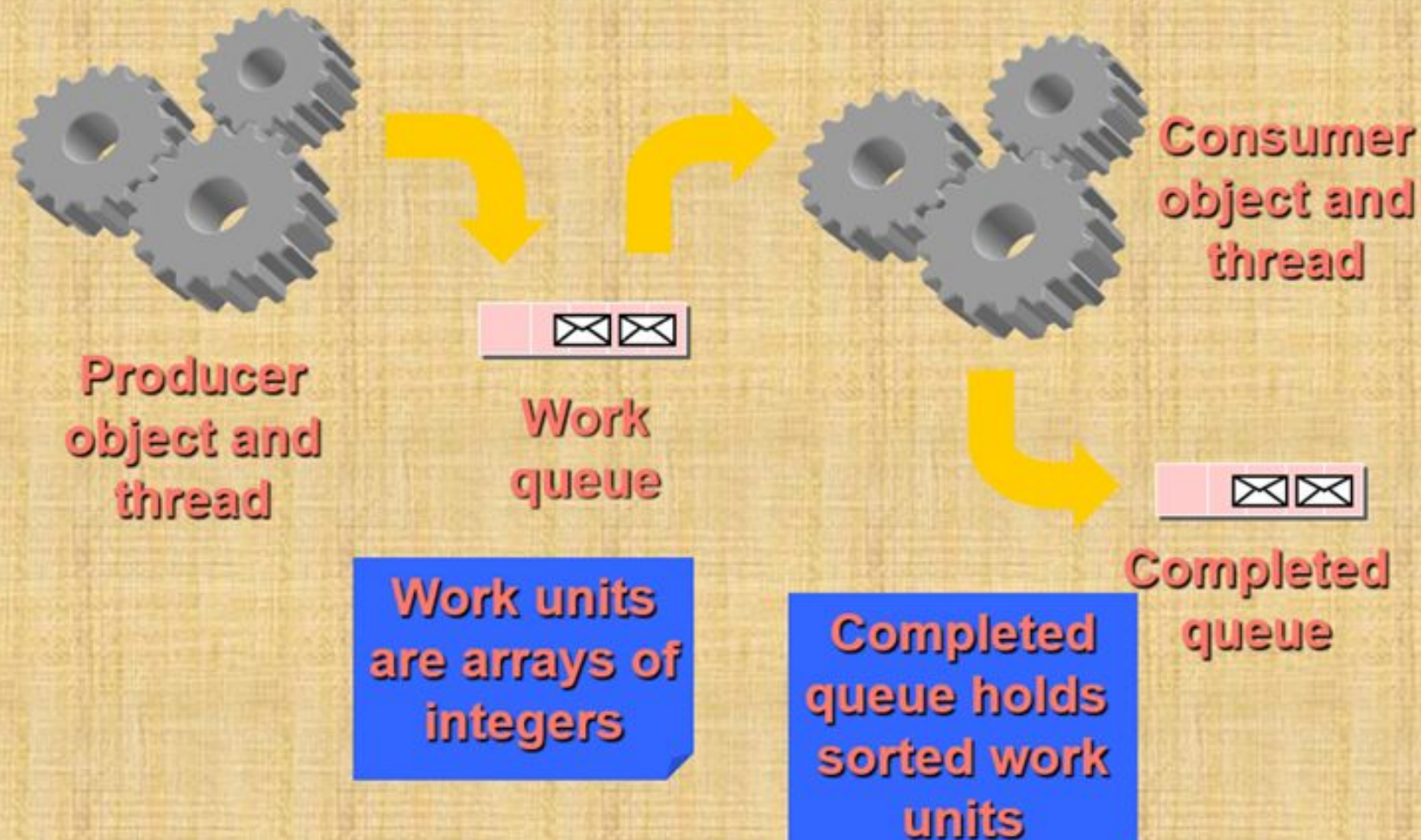
MULTITAREA



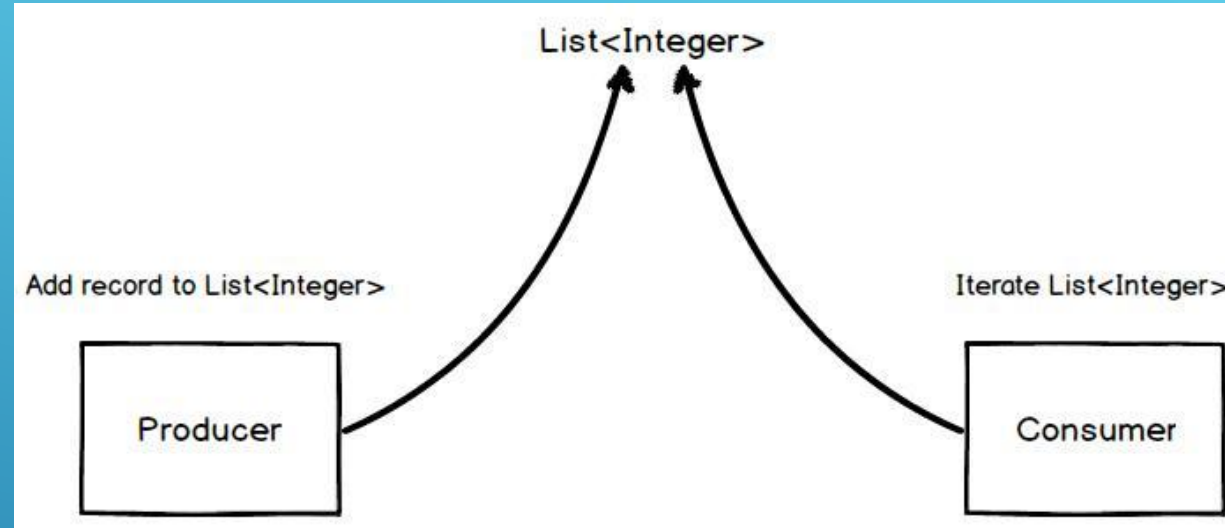
Ing. Matias Macrino

PATRÓN PRODUCTOR-CONSUMIDOR

Multi-threading



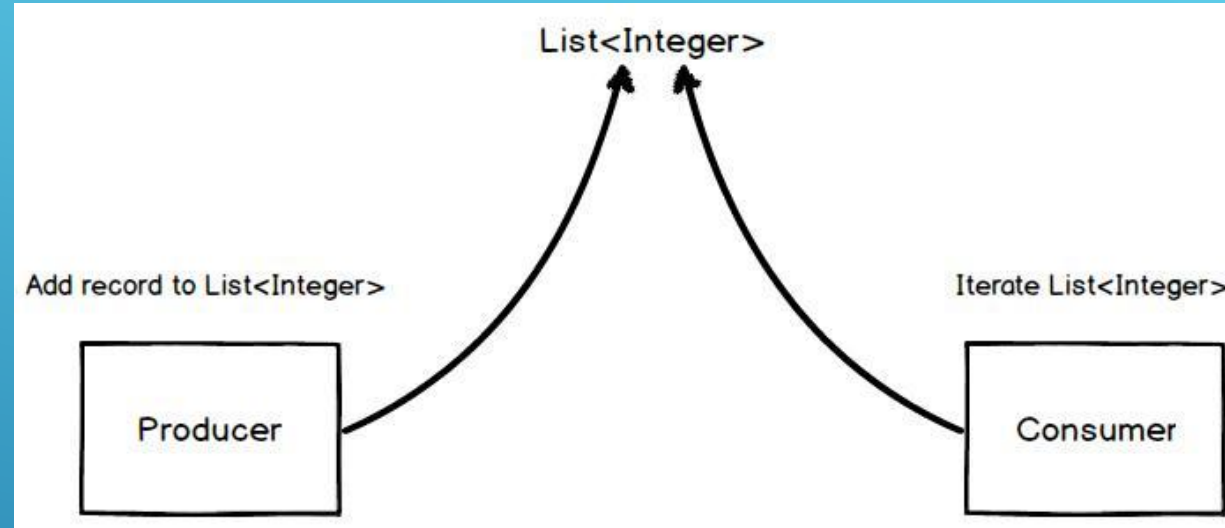
PATRÓN PRODUCTOR-CONSUMIDOR



¿Qué problema conlleva este patrón?

Dos Procesos intentan acceder al mismo recurso (Comúnmente compartido) al mismo tiempo y uno no puede realizar su operación porque otro proceso está usando ese recurso.

PATRÓN PRODUCTOR-CONSUMIDOR



```
public class ProducerConsumerExample {  
  
    /* Crear un listado de enteros "común"*/  
    private static List<Integer> listIntegers = new ArrayList<Integer>();
```

PATRÓN PRODUCTOR-CONSUMIDOR

```
public static class ProducerThread extends Thread{
```

```
    @Override
```

```
    public void run() {
```

```
        int index = 0;
```

```
        while(true){
```

```
            try {
```

```
                Thread.sleep(1);
```

```
            } catch (Exception e) {
```

```
                e.printStackTrace();
```

```
            }
```

```
            if(listIntegers.size() < 100){
```

```
                listIntegers.add(index);
```

```
                System.out.println("Producer Thread: \n" + listIntegers);
```

```
                index++;
```

```
            }else{
```

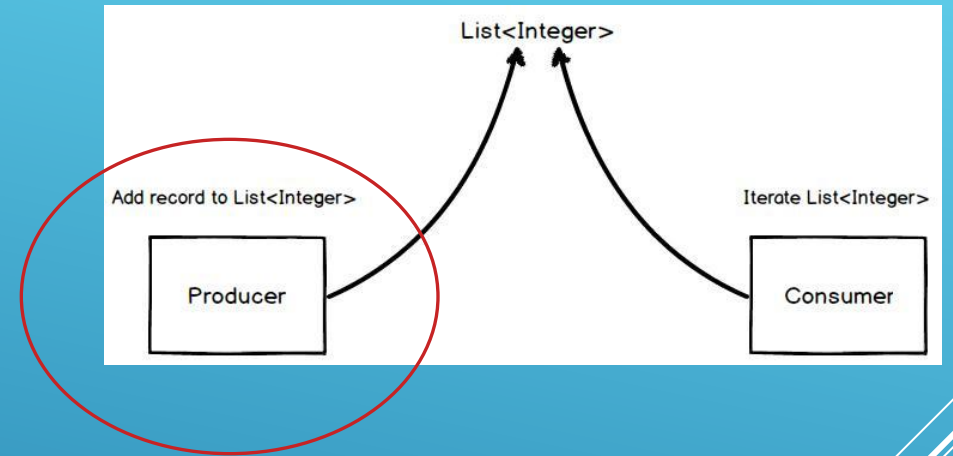
```
                listIntegers.remove(0);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



PATRÓN PRODUCTOR-CONSUMIDOR

```
public static class ConsumerThread extends Thread{
```

```
    @Override
```

```
    public void run() {
```

```
        while(true){
```

```
            try {
```

```
                Thread.sleep(1);
```

```
            } catch (Exception e) {
```

```
                e.printStackTrace();
```

```
            }
```

```
            System.out.println("Consumer Thread iterator list: ");
```

```
            Iterator<Integer> iterator = listIntegers.iterator();
```

```
            while(iterator.hasNext()){
```

```
                System.out.print(iterator.next() + ", ");
```

```
                if(!iterator.hasNext()){
```

```
                    System.out.println("");
```

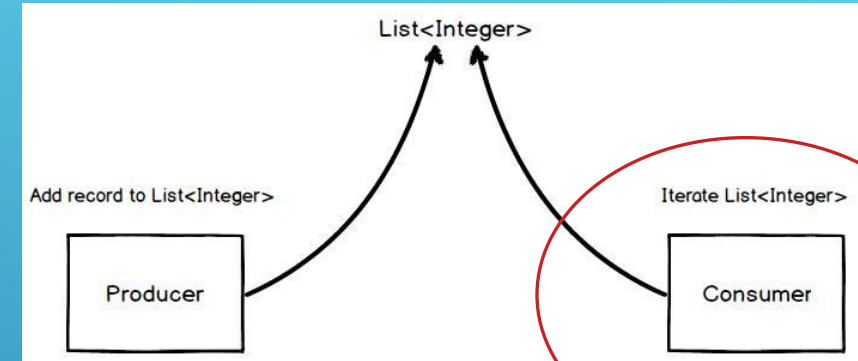
```
                }
```

```
            }
```

```
        }
```

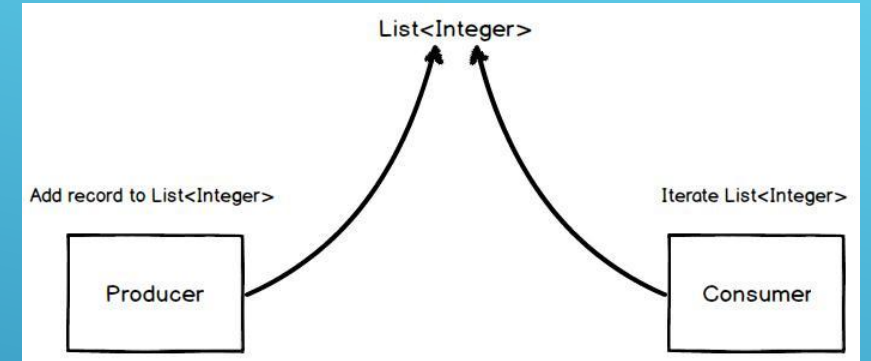
```
    }
```

```
}
```



PATRÓN PRODUCTOR-CONSUMIDOR

```
public static void main(String[] args) {  
    /* Lanzar el hilo Productor */  
    new ProducerThread().start();  
    /* Lanzar el hilo Consumidor */  
    new ConsumerThread().start();  
}
```

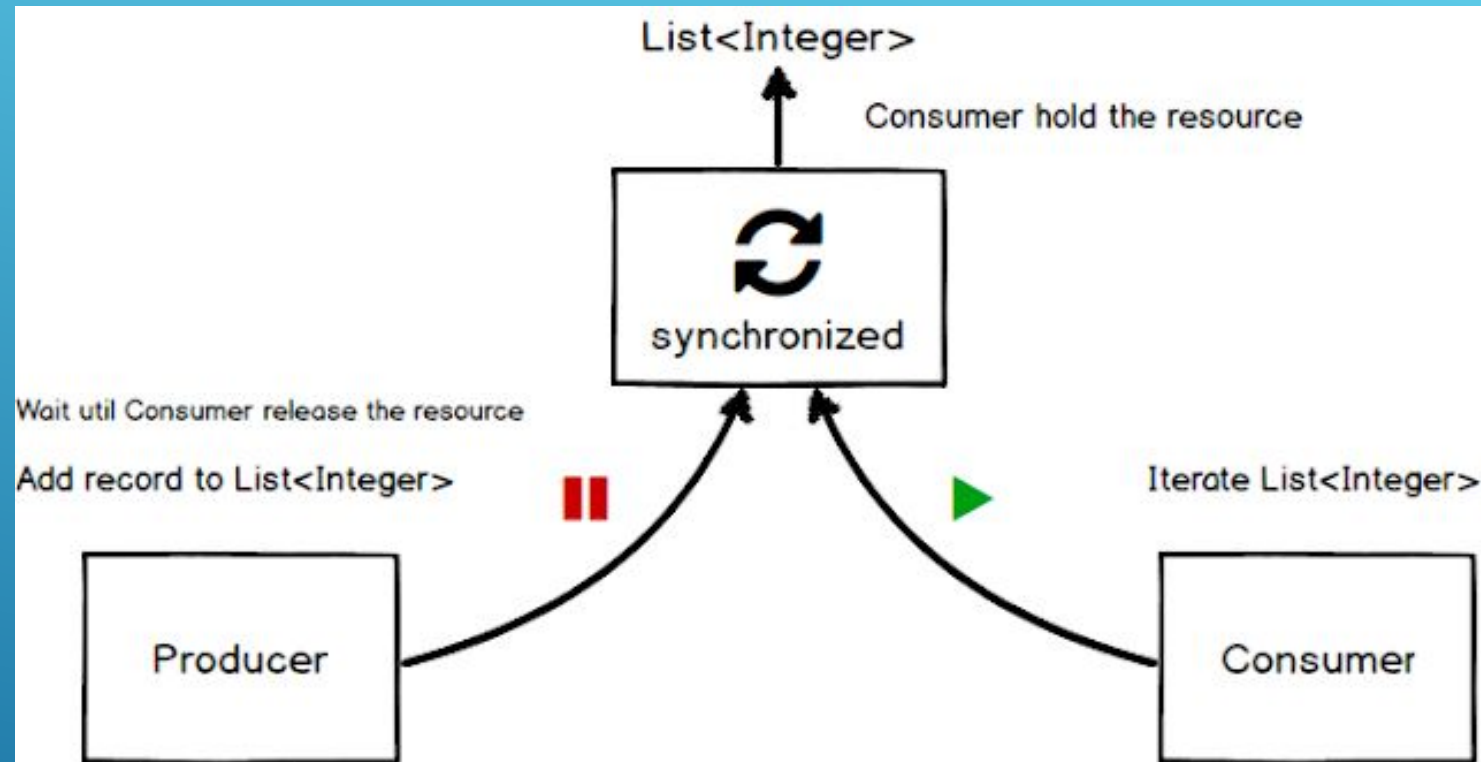


EXCEPTION

```
java.util.ConcurrentModificationException  
at java.util.ArrayList$Itr.checkForComodification(Unknown Source)  
at java.util.ArrayList$Itr.next(Unknown Source)  
at javaQuery.thread.ProducerConsumerExample$ConsumerThread.run(ProducerConsumerExample.java:51)
```

PATRÓN PRODUCTOR-CONSUMIDOR

SOLUCIÓN



Podemos resolver este problema utilizando la palabra clave **synchronize**. Mientras que un subproceso está accediendo al **List<Integer>**, otro subproceso tiene que esperar hasta que el primer subproceso termine su trabajo.

PATRÓN PRODUCTOR-CONSUMIDOR

```
public static class ProducerThread extends Thread{
```

```
@Override
```

```
public void run() {
```

```
    int index = 0;
```

```
    while(true){
```

```
        try {
```

```
            Thread.sleep(1);
```

```
        } catch (Exception e) {
```

```
            e.printStackTrace();
```

```
        }
```

```
        synchronized (listIntegers) {
```

```
            if(listIntegers.size() < 100){
```

```
                listIntegers.add(index);
```

```
                System.out.println("Producer Thread: \n" + listIntegers);
```

```
                index++;
```

```
            }else{
```

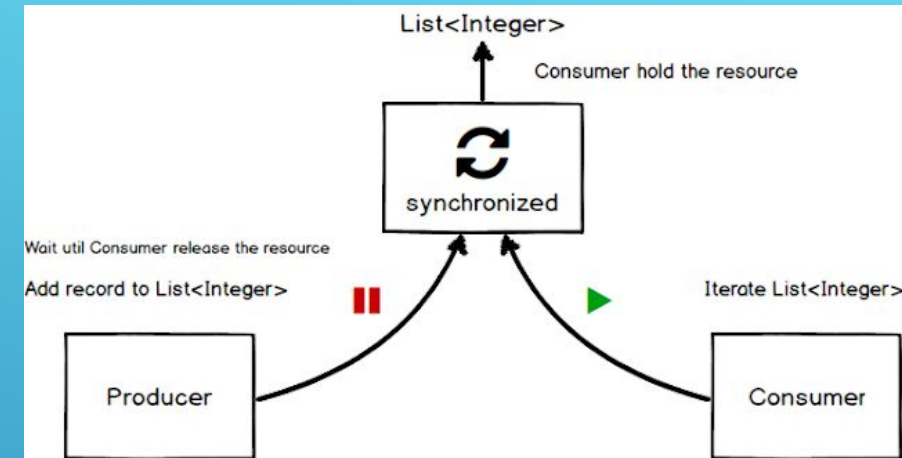
```
                listIntegers.remove(0);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



PATRÓN PRODUCTOR-CONSUMIDOR

```
public static class ConsumerThread extends Thread{
```

```
@Override
```

```
public void run() {
```

```
    while(true){
```

```
        try {
```

```
            Thread.sleep(1);
```

```
        } catch (Exception e) {
```

```
            e.printStackTrace();
```

```
        }
```

```
        System.out.println("Consumer Thread iterator list: ");
```

```
        synchronized (listIntegers) {
```

```
            Iterator<Integer> iterator = listIntegers.iterator();
```

```
            while(iterator.hasNext()){
```

```
                System.out.print(iterator.next() + ", ");
```

```
                if(!iterator.hasNext()){
```

```
                    System.out.println("");
```

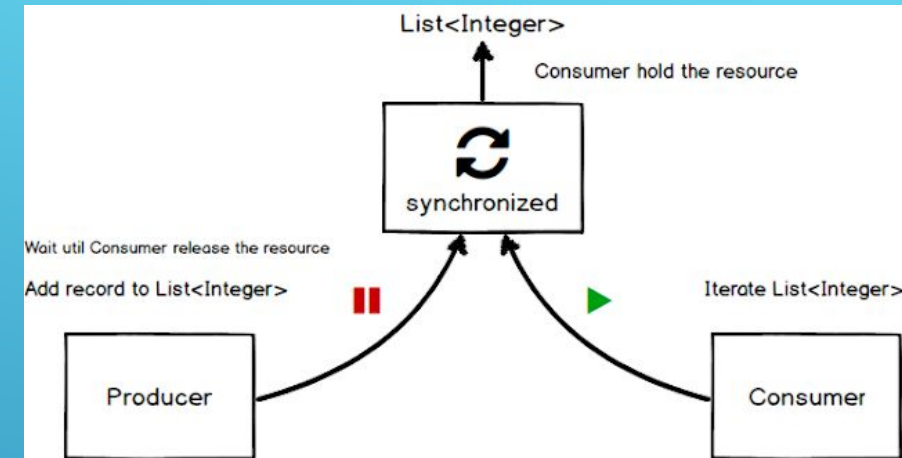
```
                }
```

```
            }
```

```
        }
```

```
    }
```

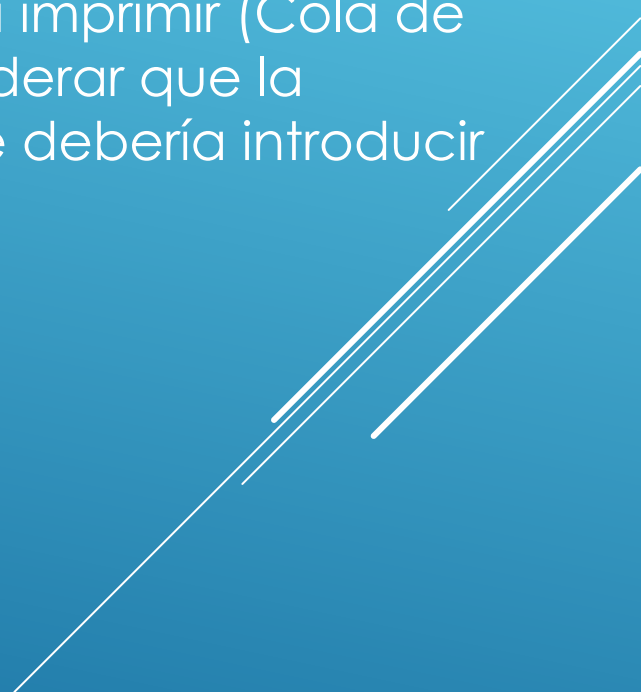
```
}
```



PATRÓN PRODUCTOR-CONSUMIDOR

Ejercitación:

Desarrollar una aplicación que simule la generación de archivos para imprimir (Cola de impresión) y una Printer que vaya imprimiendo dichos archivos. Considerar que la generación es mucho más rápida que la impresión. Qué concepto se debería introducir para solucionar el inconveniente del buffer de impresión saturado?

Several white lines of varying lengths and slopes are drawn in the bottom right corner of the slide, creating a modern, abstract graphic element.

EJERCICIO INTEGRADOR DE HILOS

Ejercitación:

- 1) Desarrollar una aplicación que simule la atención de peticiones de canales de chat. Cada canal podrá tener hasta 10 usuarios conectados a la vez. Si un cliente nro 11 intentara acceder a un determinado canal, devolver error de “Canal Completo”. Aceptar peticiones nuevamente a medida que se vacía un determinado canal de chat.
- 2) Permitir a dos clientes dialogar entre sí a través de mensajes de texto simples
- 3) Permitir a un cliente enviar un mensaje general en alguno de los canales suscriptos